

Shaders and Models

Introduction to Computer Graphics - Exercise 03

Pirmin Pfeifer

Lets have a look at our shaders:

```
1  #version version_number
2  in type in_variable_name;
3  in type in_variable_name;
4
5  out type out_variable_name;
6
7  uniform type uniform_name;
8
9  void main()
10 {
11     // process input(s) and do some weird graphics stuff
12     ...
13     // output processed stuff to output variable
14     out_variable_name = weird_stuff_we_processed;
15 }
```

We have three different global variable types:

Input variables (read from the vertex buffer or interpolated from the vertex shader output)

```
1  in vec3 position;  
2  in vec3 normal;  
3  in vec2 tex;
```

Output variables (interpolated in the fragment shader)

```
1  out vec3 normal;  
2  out vec2 tex;
```

Uniform variables (Uniform throughout the draw call)

```
1  uniform float totalTimeMs;  
2  uniform float deltaTimeMs;
```

GLSL - in and out

```
1  #version 330 core // VERTEX SHADER
2  // the position variable has attribute position 0
3  layout (location = 0) in vec3 aPos;
4  layout (location = 1) in vec3 aCol;
5
6  // specify a color to pass to the fragment shader
7  out vec4 vertexColor;
8
9  void main()
10 {
11     // see how we directly give a vec3 to vec4's constructor
12     gl_Position = vec4(aPos, 1.0);
13     // pass aCol to fragment shader
14     vertexColor = vec4(aCol, 1.0);
15 }
```

File: default.vert

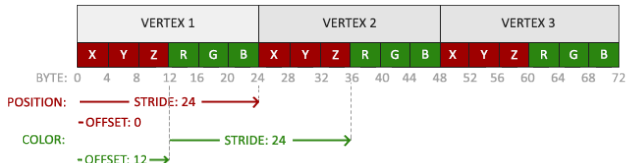
```
1  #version 330 core // FRAGMENT SHADER
2  out vec4 FragColor;
3
4  // the in variable from the vertex shader (same name and same type)
5  in vec4 vertexColor;
6
7  void main()
8  {
9      FragColor = vertexColor;
10 }
```

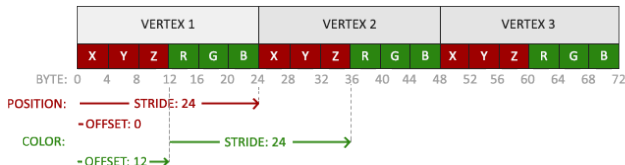
File: default.frag

We also need to adjust our vertex data:

```
1  float vertices[] = {
2      // position      color
3      0.5f,0.5f,0.0f,   1.0f,0.0f,0.0f, // top right
4      0.5f,-0.5f, 0.0f, 0.0f,1.0f,0.0f, // bottom right
5      -0.5f,-0.5f,0.0f, 0.0f,0.0f,1.0f, // bottom left
6      -0.5f,0.5f,0.0f,   1.0f,1.0f,1.0f // top left
7  };
```

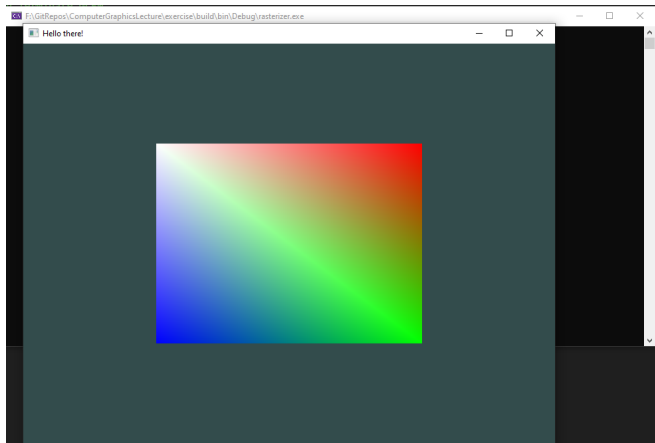
File: main.cpp





```
1 // 4. then set the vertex attributes pointers
2 glVertexAttribPointer(
3     0, 3, GL_FLOAT, GL_FALSE,
4     6 * sizeof(float), (void*)0);
5 glEnableVertexAttribArray(0);
6 glVertexAttribPointer(
7     1, 3, GL_FLOAT, GL_FALSE,
8     6 * sizeof(float), (void*)(3 * sizeof(float)));
9 glEnableVertexAttribArray(1);
```

GLSL - in and out



Uniforms are another way to pass data from the CPU to the shaders on the CPU

- ▶ globally passed to a shader program (Vertex + Fragment shader)
- ▶ the location of a uniform on a shader program can be queried with:

```
glGetUniformLocation(shaderProgram, "ourUniformName")
```

- ▶ uniforms are only compiled into the shader if they are used
- ▶ uniforms are set with the `glUniform` macros:
 - `1f 2f 3f 4f` for 1-4 floats passed individually
 - `1i 2i 3i 4i` for 1-4 ints passed individually
 - `1ui 2ui 3ui 4ui` for 1-4 unsigned ints passed individually
 - `1ui 2ui 3ui 4ui` for 1-4 unsigned ints passed individually
 - a `v` can be appended to pass a vector/array
 - there are also macros to pass matrices

Lets blend our colors to white over time:

```
1  #version 330 core // VERTEX SHADER
2  // the position variable has attribute position 0
3  layout (location = 0) in vec3 aPos;
4  layout (location = 1) in vec3 aCol;
5  // specify a color to pass to the fragment shader
6  out vec3 vertexColor;
7
8  void main()
9  {
10     // see how we directly give a vec3 to vec4's constructor
11     gl_Position = vec4(aPos, 1.0);
12     // pass aCol to fragment shader
13     vertexColor = aCol;
14 }
15
16 #version 330 core // FRAGMENT SHADER
17 out vec4 FragColor;
18
19 // the in variable from the vertex shader (same name and same type)
20 in vec3 vertexColor;
21 uniform float strength;
22 void main()
23 {
24     FragColor = vec4(mix(vertexColor,vec3(1,1,1),strength), 1.0f);
25 }
```

File: default.vert (upper half) default.frag (lower half)

We add the strength uniform and used it with mix to blend between vertex color and white:

```
1  #version 330 core // FRAGMENT SHADER
2  out vec4 FragColor;
3
4  // the in variable from the vertex shader (same name and same type)
5  in vec3 vertexColor;
6
7  uniform float strength;
8  void main()
9  {
10     FragColor = vec4(mix(vertexColor,vec3(1,1,1),strength), 1.0f);
11 }
```

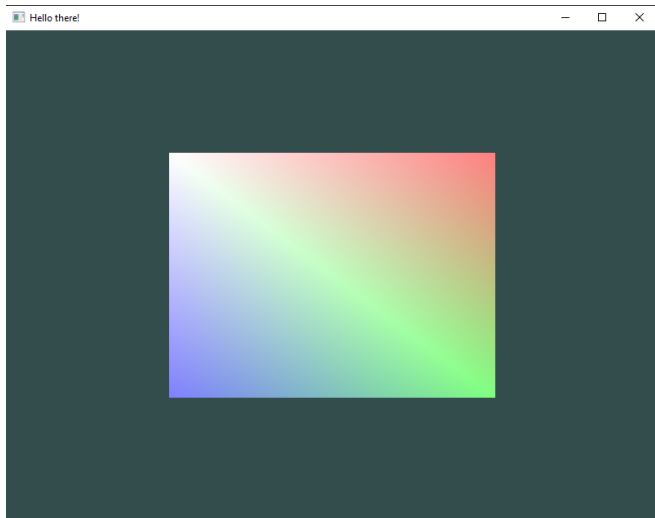
File: default.frag

In the render loop we update the uniform:

```
1
2 // be sure to activate the shader
3 glUseProgram(shaderProgram);
4
5 // update the uniform color
6 float timeValue      = glfwGetTime();
7 float strength       = cos(timeValue) / 2.0f + 0.5f;
8 int  vertexColorLocation =
9     glGetUniformLocation(shaderProgram, "strength");
10 glUniform1f(vertexColorLocation, strength);
11
12 glBindVertexArray(VAO);
13 glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
```

File: main.cpp (render loop)

GLSL - uniform Animated!



Refactoring - Shader!

Currently we have every thing in our main function, which dose not scale well, if we want to add multiple shaders and meshes

The Solution: A Shader Class!

```
1  class Shader {
2  public:
3      // constructor reads and builds the shader
4      Shader(const std::string_view vertexShaderName,
5             const std::string_view fragmentShaderName);
6      // deletes the shaders
7      ~Shader();
8
9      // use/activate the shader
10     void use();
11     // utility uniform functions we can expand them later as we need them
12     void setBool(const std::string& name, bool value) const;
13     ...
14
15     // getFunctions
16     unsigned int GetProgramId() const;
17     ...
18 private:
19     // the program ID
20     unsigned int ID;
21     unsigned int vertexID;
22     unsigned int fragmentID;
23 };
```

File: Shader.h

Refactoring - Mesh!

Currently we have every thing in our main function,
which dose not scale well, if we want to add multiple shaders and meshes

The Solution: A Mesh Class!

```
1  class Mesh {
2  public:
3      // specify vertex attributes
4      struct Attribute {
5          uint32_t type; // GL_FLOAT GL_INT etc.
6          uint32_t count;
7      };
8
9      // create mesh
10     Mesh(const std::vector<float>& vertexData,
11          const std::vector<uint32_t>& indexData,
12          const std::vector<Attribute>& vertexLayout);
13     // release buffers
14     ~Mesh();
15
16     // draw mesh (using current shader)
17     void Draw();
18 private:
19     uint32_t vertexBuffer;
20     uint32_t indexElementBuffer;
21     uint32_t vertexAttributeObject;
22     int32_t triangleCount;
23 };
```

File: Mesh.h

Loading a Model - GLTF

glTF (Graphics Library Transmission Format) is a standard file format for three-dimensional scenes and models. For loading of these files we use the tinyglTF model loader.

```
class Model {
public:
    Model() = default;
    DEFAULT_METHODS(Model)

    bool operator==(const Model &) const;

    std::vector<Accessor> accessors;
    std::vector<Animation> animations;
    std::vector<Buffer> buffers;
    std::vector<BufferView> bufferViews;
    std::vector<Material> materials;
    std::vector<Mesh> meshes;
    std::vector<Node> nodes;
    std::vector<Texture> textures;
    std::vector<Image> images;
    std::vector<Skin> skins;
    std::vector<Sampler> samplers;
    std::vector<Camera> cameras;
    std::vector<Scene> scenes;
    std::vector<Light> lights;
    std::vector<AudioEmitter> audioEmitters;
    std::vector<AudioSource> audioSources;

    int defaultScene{-1};
    std::vector<std::string> extensionsUsed;
    std::vector<std::string> extensionsRequired;

    Asset asset;

    Value extras;
    ExtensionMap extensions;

    // Filled when SetStoreOriginalJSONForExtrasAndExtensions is enabled.
    std::string extras_json_string;
    std::string extensions_json_string;
};
```

File: tiny_gltf.h (imported library)

- ▶ The model provides all scene Information encoded in the file
- ▶ For now we are only interested in the geometry. So we need:
 - Mesh info (and the primitives)
 - Accessors
 - Buffers
 - BufferViews

Loading a Model - GLTF - Mesh

```
struct Mesh {  
    std::string name;  
    std::vector<Primitive> primitives;  
    std::vector<double> weights; // weights to be applied to the Morph Targets  
    ExtensionMap extensions;  
    Value extras;  
  
    // Filled when SetStoreOriginalJSONForExtrasAndExtensions is enabled.  
    std::string extras_json_string;  
    std::string extensions_json_string;  
  
    Mesh() = default;  
    DEFAULT_METHODS(Mesh)  
    bool operator==(const Mesh &) const;  
};
```

File: tiny_gltf.h (imported library)

A Mesh contains one or multiple primitives and additional Information for animation and morphing.

Loading a Model - GLTF - Primitive

```
struct Primitive {  
    std::map<std::string, int> attributes; // (required) A dictionary object of  
                                           // integer, where each integer  
                                           // is the index of the accessor  
                                           // containing an attribute.  
    int material{-1}; // The index of the material to apply to this primitive  
                     // when rendering.  
    int indices{-1}; // The index of the accessor that contains the indices.  
    int mode{-1};    // one of TINYGLTF_MODE_***  
    std::vector<std::map<std::string, int> > targets; // array of morph targets,  
    // where each target is a dict with attributes in ["POSITION", "NORMAL",  
    // "TANGENT"] pointing  
    // to their corresponding accessors  
    ExtensionMap extensions;  
    Value extras;  
};
```

File: tiny_gltf.h (imported library)

The GLTF-Primitive corresponds to the Mesh class we defined our self:

- ▶ **attributes** - vertex attributes of the mesh
- ▶ **indices** - index Information
- ▶ **mode** - should contain **TINYGLTF_MODE_TRIANGLES**

All of the Information is saved in the buffers which are accessed through buffer views and accessors

```
struct Accessor {  
    int bufferView{-1}; // optional in spec but required here since sparse  
                        // accessor are not supported  
    std::string name;  
    size_t byteOffset{0};  
    bool normalized{false}; // optional.  
    int componentType{-1}; // (required) One of TINYGLTF_COMPONENT_TYPE_***  
    size_t count{0}; // required  
    int type{-1}; // (required) One of TINYGLTF_TYPE_*** ..  
    Value extras;  
    ExtensionMap extensions;
```

File: tiny-gltf.h (imported library)

The GLTF-Accessors contains information about how the buffer data is to be interpreted:

- ▶ **componentType** - type of the individual components
- ▶ **count** - element count (vertex/index)
- ▶ **type** - type of the elements (vec3,mat3,...)

Loading a Model - GLTF - BufferView

```
struct BufferView {  
    std::string name;  
    int buffer{-1};           // Required  
    size_t byteOffset{0};     // minimum 0, default 0  
    size_t byteLength{0};     // required, minimum 1. 0 = invalid  
    size_t byteStride{0};     // minimum 4, maximum 252 (multiple of 4), default 0 =  
                                // understood to be tightly packed  
    int target{0};           // ["ARRAY_BUFFER", "ELEMENT_ARRAY_BUFFER"] for vertex indices  
                                // or attribs. Could be 0 for other data  
    Value extras;  
    ExtensionMap extensions;  
};
```

File: tiny_gltf.h (imported library)

The GLTF-BufferViews defines which part of the buffer should be read:

- ▶ **buffer** - id of the buffer which should be read
- ▶ **byteOffset** - reading starts at that offset
- ▶ **byteLength** - number of bytes to read
- ▶ **byteStride** - how much distance is between elements

Now we can write our loading function:

```
Mesh loadFirstPrimitiveFromGltf (std::string_view modelName, bool loadNormals = true, bool loadTexCoords = true);
```

We limit the function to only load:

- ▶ The primitive from the first mesh
- ▶ index values
- ▶ "POSITION" - vertex positions
- ▶ "NORMAL" - vertex normals (optional)
- ▶ "TEXCOORD_0" - vertex texture coordinates (optional)

File: Mesh.h (definition) Mesh.cpp (implementation)

Naturally we need to update our shader to work with the vertex attributes from the model:

```
1  #version 330 core
2  layout (location = 0) in vec3 aPos;
3  layout (location = 1) in vec3 aNormal;
4  layout (location = 2) in vec2 aTexCoord;
5
6  out vec3 vertexColor;
7  out vec2 TexCoord;
```

File: default.vert

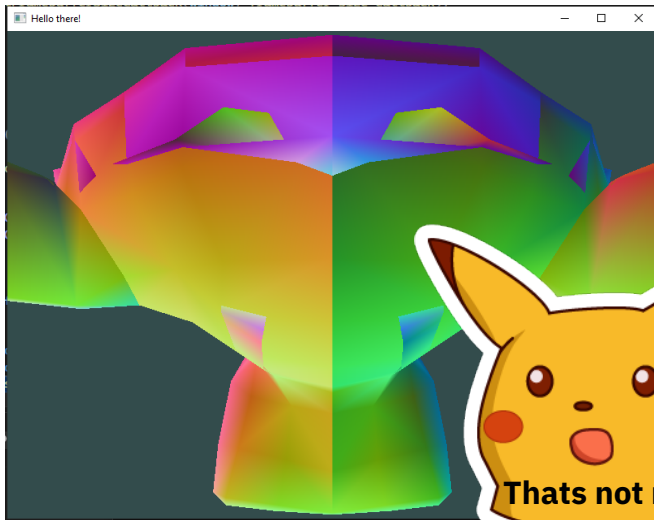
For example we now can visualize the normals or texture coordinates:

```
1  #version 330 core
2  out vec4 FragColor;
3
4  in vec3 vertexColor;;
5  in vec2 TexCoord;
6
7  uniform float strength;
8
9  void main()
10 {
11     FragColor = vec4(mix(
12         vertexColor,
13         vec3(TexCoord.xy,0),
14         strength), 1.0f);
15 }
```

File: default.frag

Rendering a Model

And there we have it:



We now render different triangles over each other, but we forgot to tell the GPU how to resolve the depth so it just goes in triangle index order:

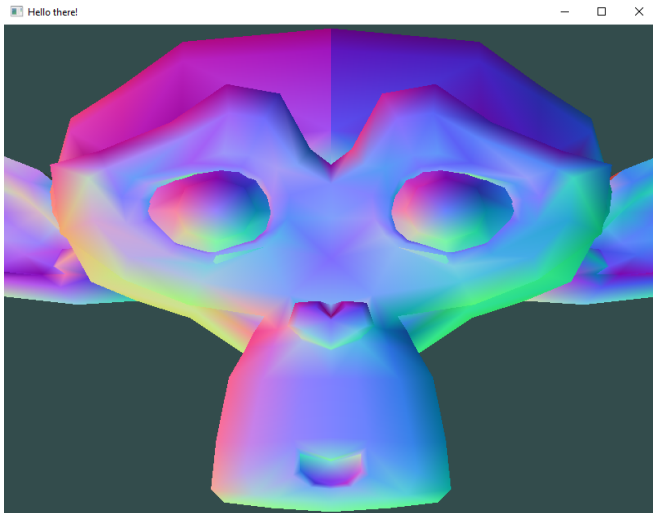
The Solution: Turn on the depth test!

```
1  glEnable(GL_DEPTH_TEST);
2  glDepthFunc(GL_LESS);
3
4  ...
5
6  // clear color and depth buffer at the start of new frame
7  glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
8  glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

File: main.cpp (main function & render loop)

Rendering a Model

Now Suzan renders correctly with Normals:



Rendering a Model

Now Suzan renders correctly with Normals and TextureCoordinates:

